

EEP3020 | Digital Systems Lab
Indian Institute of Technology Jodhpur
Supervisor: Prof. Binod Kumar

Real-Time Physical Activity Recognition on STM32F429I-DISC1

Using On-Board Random Forest Inference

Group Members

Name	Roll Number
Sohom Sarkar	B23EE1099
Sambhav Jha	B23EE1092
Rudra Khokhani	B23EE1036
Tula Mrudhul	B23EE1076

April 2026

1. Introduction

Physical Activity Recognition (PAR) classifies a person’s physical activity from inertial sensor data, with applications in health monitoring, fall detection, and context-aware computing.

We build a **complete, self-contained** system on the **STM32F429I-DISC1**: all signal conditioning, feature extraction, and machine-learning inference run on the ARM Cortex-M4F at 72 MHz with no host computer required during deployment.

The system recognises four activities: **STATIONARY** (sitting or standing still), **WALKING**, **CLIMBING** (ascending or descending stairs), and **LAYING**. A Random Forest trained on labelled inertial data collected with the same MPU-6050 hardware is compiled to portable C and embedded in flash. One activity decision is produced every 1.28 s and shown on the board’s 2.4-inch colour TFT LCD (Figure 1). All inference runs from on-chip resources only; no host computer or network connection is required after programming.



Figure 1: Completed system displaying *LAYING* at 100% confidence. Rainbow wires connect the MPU-6050 breakout via I²C3.

2. Hardware Platform

Table 1: Hardware components.

Component	Detail
MCU	STM32F429ZIT6, ARM Cortex-M4F, 72 MHz, 2 MB Flash, 256 KB SRAM
IMU	InvenSense MPU-6050 breakout (± 2 g, ± 250 °/s)
Bus	I ² C3, SCL: PA8, SDA: PC9
Display	ILI9341, 2.4 in TFT, 240×320 px
UART	USART1 @ 115 200 baud
Toolchain	STM32CubeIDE 2.1.1, GCC 14.3 arm-none-eabi

Axis remapping. Training data was collected with the board held flat, approximating a smartphone in a trouser pocket. The sensor axes are remapped at ingestion so the feature extractor sees the same coordinate frame as during training: `PAR_PushSample(raw.az, raw.ay, raw.ax, raw.gz, raw.gy, raw.gx)`.

3. System Architecture

The firmware uses two strictly separated execution contexts. The **TIM7 ISR** is the sole real-time path: it reads six IMU registers over I²C, runs the full signal-conditioning chain, and every 64th call triggers feature extraction and Random Forest inference, completing in under 200 μ s. A single **FreeRTOS LCD task** polls the pipeline output at ~ 5 Hz via a *volatile* shared struct and refreshes the ILI9341 display; its `osDelay(200)` yield ensures the ISR is never delayed by display I/O.

Table 2: Execution contexts and timing budgets.

Context	Rate	Budget	Responsibility
TIM7 ISR	50 Hz	<200 μ s	IMU read, IIR filter, body-acc, circular buffer; RF inference every 64th tick
LCD Task (FreeRTOS)	$\sim$ 5 Hz	<10 ms	Read pipeline output via shared struct, refresh ILI9341

4. Signal Processing Pipeline

4.1 Sampling and Buffering

Each sample is stored in a **9-channel circular buffer** of depth 128 (= 2.56 s), holding noise-filtered accelerometer, noise-filtered gyroscope, and body acceleration (3 axes each). Every 64 new samples (1.28 s hop, 50% overlap) a window is extracted for processing.

4.2 IIR Filtering and Body Acceleration

Two causal second-order Butterworth biquads decompose the accelerometer signal: (1) a **noise LPF at 18 Hz** applied to all six IMU axes, and (2) a **gravity LPF at 0.3 Hz** applied to the noise-filtered accelerometer output. Body acceleration is their difference, $\mathbf{b}[n] = \mathbf{a}_{\text{noise}}[n] - \mathbf{a}_{\text{gravity}}[n]$, isolating inertial dynamics and removing the ~ 1 g gravity bias.

4.3 Gain and Clipping

Training data shows ~ 0.1 to 0.2 g RMS body acceleration for walking; our board produces lower magnitudes due to placement differences. A gain factor of 6.0 rescales the signal into the trained feature range, and per-sample clipping at ± 0.5 g prevents brief mechanical taps from inflating the window RMS and triggering a false motion label:

```
#define BODY_ACC_GAIN 6.0f
#define BODY_ACC_CLIP 0.5f
float bax = (nax - gx) * BODY_ACC_GAIN;
if (bax > BODY_ACC_CLIP) bax = BODY_ACC_CLIP;
if (bax < -BODY_ACC_CLIP) bax = -BODY_ACC_CLIP;
```

A sustained walking step peaks at ~ 0.34 g (below clip, fully preserved); a 3 g tap clipped to 0.5 g contributes only ~ 0.08 g RMS over 128 samples, which is below the motion threshold.

4.4 Feature Extraction, Selection and Inference

284 features are computed per window across all nine channels: time-domain (mean, std, MAD, energy, IQR, entropy, axis correlations, AR-4 coefficients) and frequency-domain via FFT (centroid, band energies, spectral entropy, peak component), plus cross-axis signal magnitude vectors. A pre-computed index mask selects the **100 most discriminative features**, z-score normalised using training-set statistics stored in flash. The normalised vector is fed to a **Random Forest** compiled to portable C decision trees, performing inference entirely on-chip.

4.5 Majority Vote

The predicted label is appended to a 3-window circular vote buffer. The reported activity is the mode of those three labels; confidence equals the agreement fraction ($1/3$, $2/3$, $3/3$).

5. Accuracy Evaluation

5.1 LCD Display

The ILI9341 LCD is refreshed at ~ 5 Hz from a FreeRTOS task. It shows the activity name (colour-coded per class), a confidence bar that changes colour (green $\geq 70\%$, orange $\geq 40\%$, red $< 40\%$), live IMU readings, and pipeline counters.

5.2 Test Procedure

Firmware test mode (`#define PAR_TEST 1`) cycles through all four classes, 30 s each, logging one CSV line per window over UART (`phase,window,predicted,confidence_pct`). The first **3 windows per phase** are discarded (`SETTLE_WIN=3`) to allow the 128-sample circular buffer and 3-window vote buffer to fill with samples from the new activity (approximately 3.84 s settling time).

5.3 Results

Table 3: Test parameters (left) and per-class accuracy (right).

Parameter	Value	Class	Corr.	Tot.	Acc.
Sample rate	50 Hz	STATIONARY	16	18	88.9%
Window / Hop	128 / 64 samples	WALKING	18	19	94.7%
Vote depth	3 windows	CLIMBING	17	19	89.5%
Gain / Clip	6.0× / ±0.5 g	LAYING	20	20	100.0%
Duration per class	30 s	OVERALL	71	76	93.4%
Settling skip	3 windows				
Valid windows	76				

	True class	Predicted class			
		STAT.	WALK	CLIMB	LAY
STAT.		16	2	0	0
WALK		0	18	1	0
CLIMB		0	2	17	0
LAY		0	0	0	20

Figure 3: Confusion matrix (76 settled windows). Green: correct. Red-tint: misclassified. CLIMBING/WALKING confusion (3 windows) accounts for the majority of errors, consistent with their similar rhythmic motion signatures.

5.4 Discussion

The dominant error source is CLIMBING/WALKING confusion (3 of 5 errors): both activities share a strong periodic leg-swing component, making them the hardest pair to separate without additional context (e.g. barometric altitude). STATIONARY loses 2 windows to WALKING, likely during the settling period at the start of the phase. LAYING achieves 100% owing to its distinct gravity-vector orientation. The four key design decisions enabling 93.4% accuracy are: **(i)** axis remapping aligns live data with the training frame; **(ii)** Gain = 6.0 compensates for sensor placement differences; **(iii)** Clip = 0.5 g prevents tap artefacts from triggering false motion labels; **(iv)** vote depth = 3 absorbs single-window transients at the cost of a 3.84 s settling period.

6. Development Journey

1. **Hardware bring-up.** I²C verified via WHO_AM_I; TIM7 set to 50 Hz (prescaler 1439, period 999 at 72 MHz).
2. **Pipeline integration.** IIR filters, circular buffer, and feature extraction ported. Initial labels were garbage; traced to missing axis remapping.
3. **Axis remapping.** Raw logs in multiple orientations compared against the training collection protocol; correct remapping identified and applied.
4. **Gain calibration.** A tune-mode logged ($b_{\text{rms}}, b_{\text{norm}}$) per window. Linear regression on the logged pairs identified GAIN = 6.0 as providing adequate walking/stationary separation margin.
5. **Clipping fix.** With GAIN = 6.0, table taps inflated the window RMS above the motion threshold. Per-sample clipping at ± 0.5 g resolved this without degrading walking detection.
6. **Activity class tuning.** Granular initial classes merged into four robust categories (STATIONARY, WALKING, CLIMBING, LAYING) with clearly distinct inertial signatures.
7. **LCD display.** BSP ILI9341 driver integrated; activity name, confidence bar (green/orange/red), live IMU readout, and counters laid out on the 240×320 screen.
8. **Accuracy test.** 120s structured test (4 classes × 30s) with UART CSV logging; analysed by `par_test_analysis.py`; 93.4% overall accuracy with CLIMBING/WALKING as the dominant confusion pair.

7. Conclusion

We demonstrated a fully self-contained real-time PAR system on the STM32F429I-DISC1 that samples a 6-DoF IMU at 50 Hz, applies causal IIR filtering, body-acceleration separation, gain ($\times 6$), and per-sample clipping (± 0.5 g), extracts 284 features per 2.56 s window, runs a Random Forest entirely in on-chip flash and SRAM, and achieves **93.4% classification accuracy** across four activity classes with a 1.28 s response period and 3.84 s settling time. The project demonstrates that edge inference at this complexity is practical on a Cortex-M4 at 72 MHz, and that careful signal conditioning is as important to deployed accuracy as the classifier itself.

8. Limitations and Future Work

8.1 Power Optimisation

The current firmware keeps the Cortex-M4F running continuously at 72 MHz with no idle mechanism. Three hardware paths are immediately exploitable.

Sleep between ticks (WFI). The TIM7 ISR completes in under 20 μ s; the remaining ~ 18 ms of each 20 ms interval burns cycles in a busy loop. Inserting `__WFI()` (Wait-For-Interrupt) between ISR exits allows the core to enter ARM Sleep mode: all peripheral clocks remain active (so TIM7 still fires), but the CPU pipeline and fetch unit halt, cutting core draw from ~ 27 mA to ≈ 6 mA. No driver changes are required, only a single instruction.

Stop mode between 64-sample hops. The MPU-6050 contains a 1 kB hardware FIFO capable of accumulating 64 samples autonomously over 1.28 s. After each window inference the MCU can: (i) flush and re-arm the MPU-6050 FIFO; (ii) configure the INT_PIN to assert a data-ready interrupt after exactly 64 samples; (iii) arm an I²C DMA burst so that FIFO readout on wake requires no CPU involvement until DMA completion; (iv) call `HAL_PWR_EnterSTOPMode()` to gate all PLLs, HSE, and HSI. In Stop mode the STM32F429 regulator switches to its low-power range ($V_{\text{core}} = 1.2$ V), dropping MCU current to ≈ 100 μ A. The MPU-6050 in gyroscope low-power mode adds ≈ 500 μ A, giving a total idle draw of ≈ 600 μ A between hops compared with ~ 95 mA running at full speed, a reduction of more than **150 $\times$** . On wake, the HSI re-locks in < 1 μ s and the PLL settles in ≈ 300 μ s, well within the 1.28 s window budget.

Dynamic clock scaling for FFT. Feature extraction includes a 128-point FFT on each of nine

channels, but the preceding IIR and buffering stages are memory-bandwidth-limited rather than compute-limited. The MCU could operate at 16 MHz (HSI direct, PLL off) for the IIR/buffer stages, switch the PLL multiplier to 72 MHz only for the FFT block, then scale back. At 16 MHz dynamic power is $\approx 4.5\times$ lower than at 72 MHz; given that the FFT occupies only $\sim 15\%$ of the window computation time, the net average clock could fall to ≈ 20 MHz equivalent.

Motion-threshold gating. The MPU-6050 MOT_THR register enables a hardware motion-detection interrupt: when accelerometer variance drops below the threshold the device stops asserting its data-ready line. The MCU can remain in Stop mode indefinitely during prolonged stationary periods and wake only on the first above-threshold event, re-arming the full pipeline. Combined with Stop mode between hops, estimated system draw during a stationary-heavy workload falls to $\approx 150\ \mu\text{A}$ (Stop mode plus MPU-6050 wake interrupt, gyroscope powered down).

8.2 Classification Accuracy

CLIMBING/WALKING confusion. Three of the five misclassifications are CLIMBING windows labelled WALKING. Both activities share a dominant periodic leg-swing component at ≈ 1.8 Hz, making them difficult to separate from inertial data alone. Fusing altitude from a BMP280 barometric sensor (available on several STM32 Discovery add-on boards) would provide a monotonic pressure signal sufficient to disambiguate ascending stairs from level-ground walking without retraining the Random Forest.

Subject generalisation. The current model was trained and evaluated on a single subject. Inertial gait signatures vary with walking speed, limb length, and exact sensor placement. Leave-one-subject-out cross-validation on a five-subject pool would quantify generalisation without additional hardware investment.

Auto-calibration at boot. The gravity LPF requires ≈ 10 s to settle from a cold start. Averaging 200 samples during a known-stationary boot period (detected via MOT_THR) would produce a per-deployment gravity offset that compensates for sensor mounting angle, tightening the STATIONARY/WALKING decision boundary at the cost of a 4 s boot-time calibration pause.

A. Pipeline Public API

```
/* par_pipeline.h -- public interface */
void      PAR_Init(void);

/* Call at exactly 50 Hz (e.g. from TIM7 ISR) */
void      PAR_PushSample(int16_t ax, int16_t ay, int16_t az,
                        int16_t gx, int16_t gy, int16_t gz);

int        PAR_GetActivity(void);          /* 0-3 or -1 (warming up) */
float      PAR_GetConfidence(void);        /* 0.0 to 1.0 */
const char *PAR_GetActivityName(int);      /* "WALKING" etc. */
void      PAR_SetTestPhase(int phase);     /* -1 disables logging */
```

Activity indices: 0 = STATIONARY, 1 = WALKING, 2 = CLIMBING, 3 = LAYING.

B. Key Compile-Time Parameters

Define	Value	Effect
BODY_ACC_GAIN	6.0	Body-acceleration scale factor
BODY_ACC_CLIP	0.5 g	Per-sample saturation limit
WINDOW_LEN	128	Window depth in samples (2.56 s)
HOP_LEN	64	Hop size, 50% overlap (1.28 s)
VOTE_LEN	3	Majority-vote buffer depth
N_FEATURES	100	Selected feature count
SAMPLE_RATE	50	IMU sample rate (Hz)
PAR_TEST	0 / 1	Enable structured accuracy test
TEST_HOLD_MS	30 000	Duration per activity phase (ms)

C. Accuracy Test Log Format

The test mode emits UART CSV enclosed in sentinel lines:

```
=== PAR TEST BEGIN ===
# phase,window,predicted,confidence_pct
0,1,0,100
0,2,0,67
0,3,1,100
...
=== PAR TEST END ===
```

Field definitions: **phase** = ground-truth class (0 to 3); **window** = 1-based index within the phase; **predicted** = pipeline output (0 to 3); **confidence_pct** = vote-buffer agreement $\times$ 100.

Post-process with:

```
python par_test_analysis.py putty.log
```

The script discards the first **SETTLE_WIN**=3 windows of each phase, builds a confusion matrix, and reports per-class accuracy and mean confidence.